```
$newArr = array_chunk($arr, 2, true);

print_r($newArr);

Output:
Array
(
    [0] => Array
        (
            [a] => Alice
            [b] => Bob
        )

    [1] => Array
        (
            [r] => Russell
        )

)
```

`array_chunk()` takes two required parameters, the array variable and the required size of each generated array, and an optional boolean flag that controls the preservation of keys. Unless told otherwise, `array_chunk()` will numerically re-index all elements in the returned arrays. `array_chunk()` returns an array containing the input data split according to the parameters.

Here we've asked for `$arr` to be split into arrays containing two elements and supplied an array of three. Note that the final array will not necessarily be the same size as the others, depending on the number of remaining elements in the original data.

Iterating Arrays

So far we've seen how to create arrays, access their elements, and make changes to their contents. Code designed to work with arrays will typically need to act upon the entire set of data that they contain. To do this, your program will need a way to access all array elements in an efficient way, and PHP provides various constructs to enable this.